

Instituição: UEFS — **Disciplina:** Aprendizado de Máquina e Ciência de Dados — **Professor:** Dr. Rodrigo Tripodi Calumby

Questão 1 - Qual comando principal para iniciar o treinamento de um modelo?

O método principal para iniciar o treinamento de um modelo usando Scikit-learn é o método `fit()`

Questão 2 - Qual comando utilizado para classificação das amostras?

Após o treinamento do modelo, o comando utilizado para realizar a classificação das amostras é o método `predict()`.

Questão 3 - Para que servem os pipelines?

Serve para encadear as várias etapas necessárias para treinar um modelo em um único objeto unificado.

Questão 4 - Como podemos usar uma árvore de decisão para realizar uma predição numérica?

Para problemas de regressão numérica, toda a estrutura básica da árvore de decisão é mantida (nó raiz, nós internos e folhas), entretanto, ao invés de utilizar métricas que levam em consideração pureza ou ganho de informação como Índice de Gini e Entropia para dividir os dados, é utilizado o Erro Quadrático Médio (MSE) para encontrar as subdivisões que resultam no menor MSE.

Questão 5 - O que são as fronteiras de decisão?

É a região que separa as diferentes classes em um espaço de características, como uma linha que separa duas classes em espaço cartesiano. As fronteiras de decisão permitem visualizar quais áreas do espaço de características são classificadas como pertencentes a cada classe, e como o modelo está tomando suas decisões de classificação.

Questão 6 - Qual comando utilizado para particionamento de dados entre treino e teste? Quais seus principais argumentos?

`train_test_split()` é o comando utilizado para particionar os dados entre treino e teste. Seus principais argumentos são: `*arrays`: o dataset a ser dividido; `test_size`: a proporção do dataset a ser alocado para o conjunto de teste; `train_size`: a proporção do dataset a ser alocada para o conjunto de treino; `random_state`: um número inteiro que controla o embaralhamento antes da divisão dos dados; `shuffle`: um booleano que indica se os dados devem ser embaralhados antes da divisão; `stratify`: um array que é usado para garantir que a divisão dos dados seja estratificada, ou seja, que a proporção de classes seja mantida em ambos os conjuntos de treino e teste.

Questão 7 - Considerando um classificador do tipo vizinhos mais próximos, como especificar quantos vizinhos devem ser considerados no processo de classificação?

O parâmetro `n_neighbors()` é responsável por especificar quantos vizinhos devem ser considerados no processo de classificação.

Questão 8 - Considerando um classificador do tipo vizinhos mais próximos, qual o impacto em se utilizar ponderação tipo uniforme ou baseada em distância?

O padrão do parâmetro de ponderação `weights` é "uniform", o que significa que cada vizinho tem o mesmo peso na classificação. Se for definido como "distance", os vizinhos mais próximos terão um peso maior do que os vizinhos mais distantes, o que pode melhorar a precisão em alguns casos, principalmente quando os dados estão desbalanceados. Portanto, é interessante testar ambos os tipos de ponderação para verificar qual apresenta melhor desempenho para o conjunto de dados em questão.

Questão 9 - Considerando o classificador do tipo Naive Bayes, como especificar as probabilidades a priori das classes diretamente, ou seja, sem computar a partir dos dados?

Podemos especificar as probabilidades a priori das classes diretamente utilizando o parâmetro `class_prior()` do classificador Naive Bayes (O GaussianNB usa `priors()`). Este parâmetro aceita uma lista de probabilidades para cada classe, onde obviamente a soma das probabilidades deve ser igual a 1.

Questão 10 - Considerando os datasets “moons”, “circles” e “linearly separable”. Qual classificador apresenta a melhor eficácia para cada um e qual apresenta a pior eficácia? Justifique.

No dataset “moons”: K-NN, RBF SVM e Gaussian Process foram os melhores classificadores, pois conseguiram capturar a complexidade do formato dos dados, classificadores lineares não conseguiram isso. O pior classificador para esse dataset foi o QDA, pois ele assume que as classes seguem uma distribuição normal.

No dataset “circles”: O melhor algoritmo foi o K-NN, pois ele é capaz de capturar a complexidade do formato dos dados, novamente os classificadores lineares não conseguiram isso, sendo que Linear SVM foi o pior classificador.

No dataset “linearly separable”: K-NN, Linear SVM, RBF SVM, Decision Tree, Random Forest, Neural Net, Adaboost e Naive Bayes apresentaram o mesmo desempenho. Os piores desempenhos foram o Linear SVM, Gaussian Process e QDA, porém, não ficaram muito atrás dos outros classificadores, pois o dataset é linearmente separável, o que favorece o desempenho de classificadores lineares. Entretanto, modelos com estruturas mais flexíveis ainda apresentaram um melhor desempenho.

Questão 11 - Considerando classificadores do tipo árvore de decisão, o que é chamado de “caminho de decisão”?

Decision Path é o caminho percorrido desde a raiz da árvore até uma folha. Ele mostra quais decisões foram tomadas em cada nó para chegar à classificação/regressão final ou um nó específico. O caminho de decisão é útil para entender como o modelo está tomando suas decisões e para identificar quais características estão sendo usadas para fazer as previsões.

Referências

BISHOP, C. M. *Pattern Recognition and Machine Learning*. [S.l.]: Springer, 2006. ISBN 9780387310732.

Scikit-learn developers. *Scikit-learn Documentation*. 2026. <<https://scikit-learn.org/stable/documentation.html>>. Accessed: 2026-05-14.